Mission-6

Starship Operational Unit: USSE Noosphere Bridge Crew (Squadron Turing, Alpha-1)

Course: Conversational AI — Winter Semester 2026/27

Module: Week 6 — The Autonomous Warp State Machine & Deliberation Loop

IDE & Tools Standard: Visual Studio Code / Jupyter | native Python (standard library only) | Modern LLMs (Gemini, ChatGPT, Claude)

GitHub repository URL: https://github.com/PRANAY0105/SQUAD_RUMELHART_MISSION-6

1. Executive summary

To act on its own, AURA needs an explicit control loop that decides, calls tools, observes the result, and knows when to stop. This project builds that loop in native Python: An AgentStateMachine with a single transition table, a Tool Protocol with a schema validator and an error-boundary dispatcher, an MDP trajectory logger with an empirical discounted-return evaluator, and a Floyd cycle detector with halting guardrails against Ship Anomaly 06. The pieces are integrated in a deliberation loop that runs against a small simulated warp-control system, with scripted and random policies standing in for AURA's language model.

2. Problem statement and mission

Problem. An autonomous agent can fail in three ways: it can act in an illegal order, it can crash or stall on a bad tool call, and it can loop forever. Ship Anomaly 06 is the third case. The mission brief names the anomaly without describing it, so this report uses a working definition: a runaway deliberation loop in warp control in which the agent repeats the same action in the same ship state without making progress.

Mission. (1) Construct a native Python AgentStateMachine core with state transitions; (2) implement a Tool Protocol schema validator and error-boundary dispatcher; (3) build an MDP trajectory logger and empirical discounted-return evaluator; (4) deploy a Floyd cycle detector and halting guardrails.

Purpose. To see how an agent loop can be made predictable, testable and safe, and how to measure its behavior as a Markov decision process.

3. Roles and deliverables

Role	Deliverable (folder)	Main files
Data Specialist	01_Data_Specialist	mdp_logger.py, evaluation_demo.py, test_mdp_logger.py, sample_trajectories.jsonl
Protocol Specialist	02_Protocol_Specialist	tool_protocol.py, test_tool_protocol.py
Security Sentinel	03_Security_Sentinel	cycle_detector.py, anomaly_06_demo.py, test_cycle_detector.py
Systems Architect	04_Systems_Architect	agent_state_machine.py, deliberation_loop.py, ship_sim.py, policies.py, run_mission.py, state_diagram.dot, tests
Full squad	repository root	ps06_solution.ipynb, README.md, requirements.txt, this report, AURA_File_Guide_M6.xlsx

4. Theoretical foundation

Topic	Key idea	Why it matters
Finite state machine	A fixed set of states and a table of legal (state, event) transitions	Makes agent behavior explicit, testable and impossible to put in an illegal order
Deliberation loop	Observe, decide, act, observe the result, repeat until a final answer	Separates thinking (policy) from doing (tools) and from control (state machine)
Tool protocol	Declared schemas; validate arguments before running; return structured errors (JSON-RPC style codes)	A bad call becomes an observation the agent can recover from instead of a crash
Markov decision process	Transitions (s, a, r, s'); discounted return $G_t = r_t + \gamma G_{t+1}$	A common yardstick for comparing policies and reward designs
Monte Carlo evaluation	Average the returns actually observed over logged episodes	Needs no model of the environment
Floyd's cycle detection	Tortoise moves 1 step, hare 2; they meet inside a cycle; two more phases give the start μ and length λ; O (1) memory	Detects repeating behavior without storing the whole history, but only for a deterministic successor function
Halting	Whether an arbitrary program halt is undecidable in general	So, guardrails combine cycle detection with step, error and time budgets

5. Design

5.1 AgentStateMachine (Systems Architect)

Seven states (IDLE, DELIBERATING, ACTING, OBSERVING, DONE, FAILED, HALTED) and nine events. One table, TRANSITIONS, holds every legal pair; firing anything else raises Invalid Transition and leaves the state unchanged. Terminal states accept only reset, and halt and fail are legal from every non-terminal state. A tool error moves ACTING to OBSERVING, so failures reach the agent as observations rather than ending the run.

From	Event	To	Meaning
IDLE	start	DELIBERATING	Begin an episode
DELIBERATING	decide_tool	ACTING	Policy chose a tool call
DELIBERATING	decide_final	DONE	Policy gave a final answer
ACTING	tool_ok / tool_error	OBSERVING	Result (or structured error) returned
OBSERVING	continue	DELIBERATING	Feed the observation back
any live state	halt / fail	HALTED / FAILED	Guard or crash
DONE, FAILED, HALTED	reset	IDLE	Start again

5.2 Tool Protocol (Protocol Specialist)

A call is {"tool": name, "arguments": {...}}. The validator checks the arguments against a declared schema: required and unexpected names, types (a boolean is not accepted as a number), enumerations, minimum and maximum, and string length. The dispatcher is an error boundary: dispatch() never raises for ordinary failures and returns a Tool Result with one of five JSON-RPC style codes: −32700 parse error, −32600 invalid request, −32601 unknown tool, −32602 invalid arguments, −32603 tool exception. Long outputs are truncated and flagged.

5.3 MDP trajectory logger and evaluator (Data Specialist)

Each step is stored as (state, action, reward, next_state, done, info) and can be saved to and loaded from JSON lines. The evaluator computes the discounted return G_0 in Horner form, the returns-to-go with $G_t = r_t + \gamma G_{t+1}$, the mean, standard deviation and standard error of G_0 over episodes, a sweep over γ, and first-visit Monte Carlo state values. The reward model is our own design:

Event	Reward	Reason
Every tool call	−1	Step cost: finish quickly
Call that changes the ship state	+2	Progress
Invalid call (unknown tool, bad arguments)	−5	Malformed action
Tool exception (precondition refused)	−3	Acted out of order
Correct final answer (warp engaged at factor ≥ 5)	+50	Goal
False success claim	−10	Dishonest or wrong finish
Halted by the guard	−20	Runaway behavior

5.4 Floyd cycle detector and halting guard (Security Sentinel)

Floyd () runs the tortoise-and-hare algorithm on any successor function. detect_in_trace () builds the observed successor map from the agent's (state, action) steps; if each node has one successor it runs Floyd, otherwise it falls back to a test for a block repeated at the end of the trace. HaltingGuard.observe() is called after every tool call and halts on a confirmed cycle (the looping block seen three times), on 40 steps, on 4 consecutive tool errors, or on an optional time budget. Two repeats are enough for a known-deterministic policy, and the setting is configurable.

5.5 Ship simulator and policies

The simulated ship has five tools: get_warp_status, run_diagnostic (warp_core, nacelles, shields), set_warp_factor (0 to 9.9), engage_warp (refused unless warp_core and nacelles have passed and the factor is set) and disengage_warp. It is deterministic. Policies are plain callables: a good scripted plan, a sloppy plan that makes three mistakes and recovers, a looping policy that generates Ship Anomaly 06, a policy that falsely claims success, and a seeded random policy. No language model is called.

6. Implementation

File	Role	Purpose
agent_state_machine.py	Systems Architect	AgentStateMachine, State enum, TRANSITIONS table, DOT export
deliberation_loop.py	Systems Architect	Observe-decide-act loop wiring machine, dispatcher, logger and guard

tool_protocol.py	Protocol Specialist	Schema validator, ToolRegistry, Dispatcher with error boundary
mdp_logger.py	Data Specialist	TrajectoryLogger, discounted return, evaluation, gamma sweep, MC state values
cycle_detector.py	Security Sentinel	floyd(), trace detection, Halting Guard
run_mission.py	Systems Architect	Five scenarios and a 200-episode random-policy evaluation
ps06_solution.ipynb	Full squad	Integrated notebook running every module and test

7. Results and verification

Run status: all code and tests were executed and the numbers below are real outputs. The environment, reward values and policies are our own simulation, so the results show how the components behave and do not predict performance with a real language model.

7.1 Unit tests

Folder	Tests passed	Covers
01_Data_Specialist	11 of 11	Hand-computed returns, recurrence, γ = 0 and 1, statistics, first-visit values, JSON-lines round trip
02_Protocol_Specialist	14 of 14	Schema rules, bool-is-not-a-number, every error code, contained exceptions, truncation
03_Security_Sentinel	18 of 18	Floyd on textbook cases and 500 random functions against brute force, guard budgets
04_Systems_Architect	18 of 18	Exhaustive (state, event) check, loop scenarios, reproducibility
Total	61 of 61	

One bug was found by the tests: the first version of floyd() reported a cycle when the sequence ended at once, because both pointers became None and compared equal. It was fixed and the random-function comparison now agrees with brute force on all 500 cases.

7.2 Scenarios

Scenario	Outcome	Halt reason	Calls	Errs	Reward	G(.95)
A good plan	SUCCESS	—	4	0	54.0	44.44

B sloppy plan, recovers	SUCCESS	—	7	3	38.0	22.79
C Anomaly 06 loop, guard on	HALTED	SHIP_ANOMALY_06_CYCLE	4	0	−22.0	−18.86
D same loop, guard off (cap 60)	HALTED	HARD_CAP	60	0	−78.0	−18.00
E false success claim	FALSE_CLAIM	—	0	0	−10.0	−10.00

In scenario B the three mistakes (an out-of-range factor, an unknown tool, and engaging too early) were each returned as structured errors and the agent recovered. In scenario C the guard confirmed the cycle on the fourth call (cycle start 1, length 1, found by Floyd). With the guard off and a cap of 200 calls the same loop ran 200 calls for a total reward of −218, against −22 with the guard on.

7.3 Floyd versus a visited set

On a sequence with a tail of 1,000,000 nodes and a cycle of 1,000, Floyd found cycle start μ = 1,000,000 and length λ = 1,000, using two pointers; a visited-set method had to store 1,001,000 nodes. Floyd's advantage is memory. For the short traces of this mission the saving is not needed, but the method is correct and cheap.

7.4 Random policy, 200 episodes (seeds 0 to 199, γ = 0.95)

Setting	Success	False claim	Halted	Length	Mean G0 ± stderr
Guard on (max 40 steps)	7	166	27	9.2	−15.86 ± 0.64
Guard off (hard cap 200)	8	192	0	10.6	−15.38 ± 0.62

With the guard on, the 27 halts were 20 cycles, 5 runs of consecutive errors and 2 step-budget stops. The guard did not help this policy: its mean return is slightly lower than without it (the difference is within about one standard error) and it halted episodes that were not loops. The random policy mostly ends by falsely claiming success (166 and 192 of 200), so its returns are dominated by that penalty. The guard pays for itself on a true deterministic loop (7.2) but is not free on stochastic policies.

A design lesson came from this table. The first guard halted on two repeats of a (state, action) pair, and it stopped 89 of the 200 random episodes as cycles, with only 3 successes. Floyd's algorithm assumes every node has exactly one successor, and a trace that looks like a function does not prove that the policy is deterministic. The guard now requires the looping block to occur three times; this cut the cycle halts to 20.

7.5 Effect of γ and a hand check

γ	Mean G0 (random, guard on)	Standard error
0.0	−2.39	0.23
0.5	−4.50	0.23
0.9	−12.60	0.43
0.95	−15.86	0.64
0.99	−19.91	1.05
1.0	−21.25	1.23

Hand check of the evaluator: for rewards [−1, −1, 10] and γ = 0.9, G_0 = −1 + 0.9· (−1) + 0.81·10 = 6.2, which is what discounted_return() prints. As γ rises, later step costs count more and the return of this poor policy falls. First-visit Monte Carlo gives V(start) = −15.86 from 200 episodes; the value of the state with both diagnostics passed and factor 6.0 is −3.99, but it rests on only 4 first visits and should not be trusted.

7.6 Verification performed

- 61-unit tests pass; the notebook’s code cells were executed end to end without errors.
- Floyd was checked against brute force on 500 random functional graphs and against textbook cases.
- All numbers in 7.2 to 7.5 are outputs of the delivered scripts (run_mission.py, anomaly_06_demo.py, evaluation_demo.py).
- Not done: the instructor spec ps06.md was not accessible, no language model was connected, and the reward values were not tuned or validated.

8. Conclusion

We now have a mall but complete agent control stack: a state machine that cannot be put in an illegal order, a tool layer that turns every failure into an observation, a logger and evaluator that put numbers on behaviour, and a guard that stops a runaway loop after four calls instead of two hundred. The experiments also show the limits: Floyd’s method needs a deterministic successor, so the guard has to be conservative with sampling policies, and no guard removes the need for step and error budgets.

- Compare the design choices (states, schema format, rewards, definition of Ship Anomaly 06) with the instructor spec ps06.md and adjust.
- Validate the reward design and compare the guard on more varied policies.

9. Squad sign-off

Station	Cadet name
Data Specialist	Ashwan Lingala
Protocol Specialist	Lakshman Kumar Tata
Security Sentinel	Bharath Reddy Vupperpally
Systems Architect	Pranay Kumar Reddy Galva